P2243R0: Language linkage for templates

Audience: EWG
S. Davis Herring <herring@lanl.gov> and Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>
September 25, 2025

Introduction

There are several kinds of language linkage that are easily confused due to the unusual syntax of the *linkage-specification*s that bestow all of them. This paper removes one of them that is specified, vaguely, solely as a restriction, resolving CWG1463.

Background

Per [dcl.link]/1 (and /5), language linkage applies to function *types* and separately to *functions and variables* (with external linkage). Language linkages other than C and C++ are "conditionally-supported, with implementation-defined semantics" (/2). However, [temp.pre]/6 uses "language linkage", without any introduction, in regards to *templates* (including, apparently, class and alias templates and concepts).

Function type

```
void f();
// cf is the type "C function of () returning void"
extern "C" typedef void cf();
cf *p=f;                        // error: type mismatch
```

This corresponds to the ABI notion of a calling convention. CWG1555 points out that several common implementations ignore this aspect of the language, accepting the above but rejecting

```
void use(cf) {}
void use(void()) {}             // error in practice: redefinition
```

This paper changes nothing in this regard, although it removes an obstacle to using such types properly with an implementation that distinguishes them.

Function or variable (with external linkage)

```
namespace N {
  typedef void* mk(int);      // prepare a type with C++ language linkage
  extern "C" mk make_vector;  // "C" applies to function, but not its type
  void* make_vector(int i)    // redeclaration retains language linkage
  {return new std::vector<int>(i);}
}
int make_vector;              // error: collides with N::make_vector
```

This corresponds to the ABI notion of a name mangling scheme; of course, this multi-step declaration is not usually needed because one would typically use a C calling convention for a function that could be named by a C program. Note that it effectively applies by default to variables in the global scope, reflecting the fact that in practice no mangling is used for them. Oddly, it applies even to operator functions:

```
struct A {};
struct B {};

extern "C" {
void operator+(A);
void operator+(B);            // error: conflicting function type
}
```

but is prohibited for user-defined literals by [over.literal]/6.

Template

The permissive syntax of *linkage-specification*s can plausibly apply to templates. The "implementation-defined semantics" rule, construed broadly, could thereby affect them, and so could the apparent intent of [temp.pre]/6:

```
// Implementation-defined validity/semantics:
extern "Java" template<class T> void f(T*);

extern "C" template<int> void unrolled(int*);  // error: C language linkage
```

The first of these might be used to implement some sort of gateway to, say, Java generics; the latter is simply forbidden by [temp.pre]/6 (if, again, we permit the notion to apply at all).

Problem

Templates have no need of `extern "C"` for name-mangling purposes, since their specializations must be mangled differently. While they can certainly use function types with C language linkage, prohibiting template declarations inside `extern "C"` interferes with doing so in two different ways.

Dependent C-linkage types

Consider a C API that can fill in several kinds of object:

```
/* getters.h */
#ifndef GETTERS_H
#define GETTERS_H

#ifdef __cplusplus
extern "C" {
#endif

struct things {unsigned a,b;};

void get_int(int*);
void get_int_fast(int*);
void get_float(float*);
void get_things(struct things*);

#ifdef __cplusplus
}
#endif

#endif
```

We might want to wrap such functions:

```
#include"getters.h"

namespace wrap {
  extern "C" typedef void (*int_getter)(int*);

  int get_int(int_getter f) {
    int ret;
    f(&ret);
    return ret;
  }
}

namespace client {
  void g() {
    auto val=wrap::get_int(get_int_fast);
    // ...
  }
}
```

The obvious next step is to generalize to

```
namespace wrap {
  template<class T>
  T get(/* ... */ f) {
    T ret;
```

```
    f(&ret);
    return ret;
  }
}
```

but we encounter difficulty writing the type for the parameter `f`. Writing `void (*f)(T)` directly produces a function type with C++ linkage because we cannot wrap it in `extern "C"`. Nor are we able to specify it as a separate alias template:

```
namespace wrap {
  extern "C"
  template<class T>
  using getter=void(*)(T);    // error: template has C language linkage

  template<class T>
  T get(getter<T> f) {
    T ret;
    f(&ret);
    return ret;
  }
}
```

It may be possible to use a trick involving a nested class:

```
template <typename T>
struct A {
  struct B;
};

extern "C" {
template <typename T>
struct A<T>::B {
  typedef void (*fpty)(T);
};
}
```

although it could be argued that the intent of [temp.pre]/6 is to prohibit this as well. This approach also prevents template argument deduction.

Function templates with C-linkage types

Consider another C API, which accepts responsibility for destroying objects:

```
/* cleanup.h */
#ifndef CLEANUP_H
#define CLEANUP_H

#ifdef __cplusplus
extern "C" {
#endif
```

```
void transfer_ownership(void*,void(void*));

#ifdef __cplusplus
}
#endif

#endif
```

We can wrap it to use destructors:

```
#include"cleanup.h"
#include<memory>

class A {/* ... */};

extern "C" void del_A(void *p) {delete static_cast<A*>(p);}
void transfer_A(std::unique_ptr<A> a) {transfer_ownership(a.release(),del_A);}
```

The generalization to any type is obvious, but we cannot write

```
extern "C" template<class T> void del(void *p) {delete static_cast<T*>(p);}
```

Since the function type here is not dependent, we can again use the multi-step declaration approach:

```
extern "C" typedef void deleter(void*);
template<class> deleter del;  // a function
template<class T> void del(void *p) {delete static_cast<T*>(p);}
```

This, however, is not a self-inconsistent case deserving of such circumlocutions. Note that the questionably valid trick with the member class of a class template wouldn't allow dependent parameter or return types here: [temp.spec.general]/8 requires functions with dependent types to be declared with a function declarator (which cannot use C linkage).

Proposal

Since the rule in [temp.pre]/6 refers to a notion of language linkage that is never defined nor otherwise referenced, it can simply be removed. The existing rules for function types apply to the declarators in template declarations, and those for functions and variables themselves do not apply to template specializations because they do not have (names with) external linkage.

The potential for, say, `extern "Java"` (which was once supported by GCC) to influence a template (or a class) is preserved by the general provision in [dcl.link]/2 that strings other than `"C"` and `"C++"` have broad "implementation-defined semantics". Note that it remains impossible to declare a function template with such a language linkage whose specializations have a dependent type with a language linkage different from that given by the extended *linkage-specification*. It might be reasonable to relax the [temp.spec.general]/8 rule in question, but the resulting multi-step declarations would remain quite obtuse and redundant. We explicitly forbid

```
template<class> extern "C++" void f() {}
```

which is already uniformly rejected by implementations to reserve it for such use cases if desired.

Note also that

```
extern "C" {
  template<class T> void f(T);
  template<class T> void g(void(T));
}
template<class T> void f(T) {}
template<class T> void g(void(T)) {}
```

declares (and defines) one function template `f` (the types of whose specializations have C language linkage) but declares two function templates `g`, one of which accepts a pointer to a C function (as well as using the C calling convention itself).

Wording

Relative to N5014.

#[temp.pre]

Change paragraph 2:

> - 1. […]
>   2. be an *alias-declaration*.
>
> It shall not be a *linkage-specification*.

Change paragraph 6:

> - A specialization (explicit or implicit) of one template is distinct from all specializations of any other template. ~~A template, an explicit specialization ([temp.expl.spec]), and a partial specialization shall not have C language linkage.~~
>
> [*Note*: Default arguments for function templates and for member functions of class templates are considered definitions for the purpose of template instantiation ([temp.decls]) and must obey the one-definition rule ([basic.def.odr]). — *end note*]